HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Quiz # 1 Solution

QUESTION 1: (20 marks)

You are given a stack of unique numbers, i.e. no number appears twice in the stack. The stack is called ***StackNum*** containing 'n' elements, where 'n' is at least 2. You are also provided with an integer variable called ***in***, which is also at least 2. Assume both 'n' and 'in' are **correctly** given to you, so you may use them directly in your algorithm, without having to do any error checking or taking input. However, do declare them in the algorithm to define what they are, including the stacks and other ADT that you may use for your program

The task is to remove elements from ***StackNum*** from position, ***in*** and all multiples of ***in***, and add them in a sorted manner (ascending order) to a new stack called ***StackIns***. For example,

- Assume, ***StackNum*** = [9, 2, 5, 11, 17, 4, 1, 8, 7, 12, 19, 13, 20] and ***in*** = 3
- This means all the elements at positions – 3, 6, 9, 12, 15, … etc. should be removed from the stack, if they exist
- In terms of array indexing, these positions will refer to the indices: 2, 5, 8, 11, 14, … etc. respectively.
- In this example then, the elements to be removed will be: 5, 4, 7, 13
- These elements need to be stored in the new stack called ***StackIns*** sorted in an ascending order, thus resulting in: ***StackIns*** = [4, 5, 7, 13]
- **PLEASE NOTE the output stack,** *StackIns* **is exactly of the same size as expected from the given example, i.e. there are no extra None values in it. Hence make sure you define your stack sizes accordingly and correctly.**

You must implement the above task by writing an algorithm.

Restrictions:

- Do not **HARD CODE** your design for the given example. The algorithm should work for any valid sized stack of numbers with any valid index value.
- Assume the number of elements in original stack is 'n', where n > 1, so there will always be at least two numbers present in the stack
- Assume index 'in' is > 1, so there will never be a case where you will be removing all elements from the ***StackNum***, which also means the size of ***StackIns*** will always be smaller than ***StackNum***.
- ***StackIns*** must NEVER have any additional slots apart from the elements required to be in it, so in case of above example, ***StackIns*** will never be something like [4, 5, 7, 13, None] or [4, 5, 7, 13, None, None] … and etc.
- Assume None represents empty slots in all the data structures used for this question
- You are allowed to use **ONLY ONE** additional **ADT**, which cannot be the ListADT
- **DO NOT USE ANY OTHER DATA STRUCTURES INCLUDING ListADT/Python lists apart from the ones permitted to use**
- **REMEMBER: Stacks and Queues are non-iterable and non-traversing data structures, so you CANNOT and MUST NOT do indexing within the data structures to access the elements**

SOLUTION:

#Required variable declarations and initialization

- n – an integer representing the number of elements in the input stack
- StackNum = [None] * n
- in – an integer representing the multiple of indices that should be extracted from the input stack, i.e. StackNum
- new_size = n//in – this is to get the exact size of the output stack, i.e. StackIns
- StackIns = [None] * new_size
- StackExtra = [None] * n

#Accessing all the elements at index "in" and its multiples on StackIns, while storing all the rest on StackExtra
#so that they can be retrieved later for reusage

- for i in range 1 to n (both inclusive), do the following:
    - element = pop from StackNum
    - if i % in == 0:
        - push element on StackIns
    - else:
        - push element on StackExtra

#Once all the elements at index "in" and its multiples are in StackIns, restore the remaining values back in
#StackNum from StackExtra

- while StackExtra is not empty, do the following:
    - element = pop from StackExtra
    - push element on StackNum

#Now let's sort the elements in StackIns
#As stacks are NOT iterable, we will need to remove one element at a time from the stack and then rearrange it in
#ascending order. As there are new_size elements in StackIns, so we run the loop new_size times

- for new_size number of times, do the following:

    #First find the minimum

    - min_elem = top of StackIns

    #As we CANNOT index into stack, we will need to take one element at a time out of the stack and check for
    #minimum. The numbers should not be discarded so use StackExtra to temporarily save them while removing
    #them to find the minimum

    - while StackIns is not empty, do the following:
        - element = pop from StackIns
        - if element <= min_elem:
            - min_elem = element
        - push element on StackExtra

    #Now that we have one minimum, we put that temporarily in StackNum, so we can restore it to StackIns
    #once all elements have been sorted at the end of the outer loop running new_size times
    #Put the remaining elements other than minimum back on StackIns, so that next minimum can be found for
    #sorting purposes them

    - while StackExtra is not empty, do the following:
        - element = pop from StackExtra
        - if element == min_elem:
            - push element on StackNum
        - else:
            - push element on StackIns

#Once the above is over, StackIns would have become empty, and StackNum will have num_size number of #elements on the top. We need to restore these back on StackIns, but if we put them directly from StackNum to #StackIns, the order will reverse, and won't be the one desired, so we temporarily shift them to StackExtra first

- for new_size number of times, do the following:
    - element = pop from StackNum
    - push element on StackExtra

#Now from StackExtra when we move them to StackIns, it will be in the desired order as asked in the question

- while StackExtra is not empty, do the following:
    - element = pop from StackExtra
    - push element on StackIns

QUESTION 2: (5 marks)

We have the following array stored in memory: ***arr1*** = [1, 2, 3, 4, 5, 6, 19, 20, 21, 22, 11, 12, 7, 9, 11]. Assume each number is 32 bits in size, and one location of memory is 8-bits wide. Calculate the address of the element, arr1[6]

SHOW ALL THE NECESSARY STEPS/FORMULA TO ANSWER THE ABOVE QUESTION, OTHERWISE, NO MARKS WILL BE AWARDED EVEN FOR CORRECT ANSWERS!

SOLUTION:

We know that for single-dimensional arrays, the address of each location is calculated as:

$$\textit{Memory Address} = \textit{Base\_Address} + i * \textit{element\_size}$$

- Assume base address = 0, i is given in the question as 6,
- What about element size?
    - It needs to be calculated based on the size of the array value, as well as the memory location size, i.e.
        - element size = 32/8 = 4
- Therefore, memory address = 0 + 6 * 4 = 24